

Software Development and Design Processes

CMPT 145

Copyright Notice

©2020 Michael C. Horsch

This document is provided as is for students currently registered in CMPT 145.

All rights reserved. This document shall not be posted to any website for any purpose without the express consent of the author.

Learning Objectives

After studying this chapter, a student should be able to:

- Describe the difference between software development and software design.
- Describe the waterfall model of software development.
- Describe the iterative method of software development.
- Apply the waterfall and iterative models to assignment-sized problems.
- Discuss the importance of version control.
- Explain the purpose of prototyping.

Why development process matters

- Your time is the most important resource you have.
- If you have a manager, your time will be managed.
- If you are working for yourself, you have to manage and plan your own time.
- Planning your work is essential to productivity.
- You cannot design and code at the same time. These are two different skills.
- Debugging your poorly planned software will take up most of your time.

Bad advice

How to **fail** at programming assignments:

- Don't read the assignment carefully.
- Start programming without a plan, and without a design.
- Start late.
- Write lots of code without testing any of it.
- Do all your testing after you've written all your code
- Assume everything will go perfectly.

Common Processes

- Development: A plan for the life-cycle of a software project
 1. Waterfall model
 2. Iterative and incremental model
 3. Test-Driven Design
- Design: A plan for one step in the development process
 1. Stepwise refinement
 2. Test-Driven Design

Waterfall model tasks

1. Requirements
 - Figure out what the software is supposed to do.
2. Design
 - A plan describing how to meet the requirements.
3. Implementation
 - Write the code using the plan.
4. System Testing and Verification
 - Show that the system meets the requirements.
5. Maintenance
 - Install, deliver, update and upgrade, etc

Waterfall model: Requirements step

- The requirements for a project are acquired (negotiated) in advance.
- A requirements document is produced.
- Written in natural human language (e.g., English)
- All parties (e.g., developers, clients) need to agree on this.

Waterfall model: Design step

- Design is a planning process.
Not an implementation process.
- A design plan describes the functions and modules needed to achieve the requirements.
 - Each function is described by a specification: inputs, outputs, assumptions.
- For us, written in natural human language (e.g., English)
 - Mathematics is often used as well.

Waterfall model

- **PRO:** Simple strategy.
- **CON:** Only works for smallish projects, like first/second year assignments.
- **CON:** In real projects, requirements often **change** (especially corrections)

Waterfall model for students

1. Requirements.
 - Study the assignment/project description.
 - Ask for clarifications as necessary.
2. Design
 - A plan describing functions, scripts, modules needed.
3. Implementation
 - Write the code (functions, scripts, modules)
4. System Testing and Verification
 - Write test cases for everything
 - Show that the system meets the requirements.
5. Maintenance
 - Tidy up, and submit.

Waterfall model deadlines for students

1. Requirements.
 - Done **within one day** of assignment release.
2. Design
 - Done **no later than halfway** between release and due date.
3. Implementation
 - This will take **4 times longer** than you planned.
 - Include time for all testing.
4. System Testing and Verification
 - Done **at least one day before** the due date.
5. Maintenance
 - Tidy up, and submit.

Example 1

Use the Waterfall model to plan the development of the **Coupon Collector's** problem.

- (a) State the requirements as clearly as possible.
- (b) Plan the design, in terms of functions, modules, data organization, and test cases.
- (c) Based on the design, estimate the time to implement the design, including test cases.
- (d) Assuming a successful implementation, estimate the time to demonstrate the system, and check the requirements were met.
- (e) Estimate the time to make the project presentable to a marker.

Example 2

Use the Waterfall model to plan the development of the **Self-Avoiding Random Walks** problem.

- (a) State the requirements as clearly as possible.
- (b) Plan the design, in terms of functions, modules, data organization, and test cases.
- (c) Based on the design, estimate the time to implement the design, including test cases.
- (d) Assuming a successful implementation, estimate the time to demonstrate the system, and check the requirements were met.
- (e) Estimate the time to make the project presentable to a marker.

Iterative and incremental model

1. Start with a minimal project with only a few of the requirements.
2. Apply the waterfall model
3. Deliver the small system, get feedback.
4. Repeat steps 2-3, adding more requirements each time.

Iterative and incremental model: pro and con

- **PRO:** Development and testing is shorter.
- **PRO:** Faster feedback is good.
- **CON:** Without careful planning, requirements added later may be inconsistent with earlier versions of the system.

Iterative and incremental model for students

- Obtain **full requirements** for project.
- Identify the **core** functionality.
- Apply the **waterfall model** to the core.
- Apply **incremental model** to the remaining requirements.
 1. Consider remaining requirements, one at a time
 2. Design the addition of the requirements.
 3. Test, test, test.
 4. Repeat steps 2-3, adding more requirements each time.
- Tidy up, and submit.

Planning and designing for programming assignments

- Identify deliverables and **requirements**.
- Identify concepts related to your **lecture material**.
- Break down larger scripts into manageable pieces, using **step-wise refinement**.
- Design and plan **test cases** that you can use to check your work as you go.
- Use your analytical skills to challenge your assumptions.
- **Turn off** anything you know distracts you from work.

Example 3

Use the Incremental and Iterative model to plan the development of the **Coupon Collector's** problem.

- (a) State the requirements as clearly as possible.
- (b) Identify a core functionality from the requirements.
- (c) Plan the design of the core functionality, in terms of functions, modules, data organization, and test cases.
- (d) Identify other requirements that could be planned, estimate the time to complete them.
- (e) Assuming a successful implementation, estimate the time to demonstrate the system, and check the requirements were met.
- (f) Estimate the time to make the project presentable to a marker.

Developing Coupon Collector

Requirements:

- The number of unique coupon types: k
- Randomly select one of the coupons, with replacement.
- Count the number N of selections are needed to observe at least one instance of all k coupons.
- Repeat the experiment several times to determine an average for N . Report the average.

Developing Coupon Collector

Core functionality:

- Generate a random integer in the range $0, k$, and record the single observation in a list.
- Test case:
 - When $k = 1$ the list has one `True` value
- Time estimate: 5 minutes.

A small core functionality was chosen. Choosing something a bit bigger is not bad, but it takes practice to choose something that's not too big to complete.

Developing Coupon Collector

Additional Requirements, iteratively:

- Loop to generate 10 random numbers; test that no more than 10 observations were recorded. 5 minutes.
- Determining that all k coupons were observed; test that the list recorded at least one observation for each coupon. 10 minutes.
- Count the number of coupons N generated by the loop; test that $N \geq k$. 5 minutes.
- Put the loop in a function `trial()`, parameter is k , return value is N . Test $k = 1$, $k = 2$, $k = 10$. 5 minutes.
- Write a loop to call the function `trial()` T times; test $T = 1$, $T = 10$, $T = 100$. 5 minutes.
- Calculate the average for N , using T . Test cases: $T = 1$, $T = 10$, $T = 100$, $T = 1000$. 5 minutes.

Developing Coupon Collector

Testing and Validation:

- Check that the program meets the requirements: The average N for $k = 100$ should vary a lot for small T , but should be relatively stable for large T . 5 minutes.
- Estimate the time to make the project presentable to a marker: 5 minutes.

Total time: 45 minutes!

Practice estimating your time costs!

- If you do not monitor your time, you cannot monitor your progress!
- If you do not practice estimating your time to complete a task, you cannot know how long something will take.

Exercise 4

Use the Incremental and Iterative model to plan the development of the **Self-Avoiding Random Walks** problem.

- (a) State the requirements as clearly as possible.
- (b) Identify a core functionality from the requirements.
- (c) Plan the design of the core functionality, in terms of functions, modules, data organization, and test cases.
- (d) Identify other requirements that could be planned, estimate the time to complete them.
- (e) Assuming a successful implementation, estimate the time to demonstrate the system, and check the requirements were met.
- (f) Estimate the time to make the project presentable to a marker.

Version Control

- Software tools to help you manage content creation, esp. software
- Each *version* is a full backup, with documentation, of the state of your project.
- Maintains a documented history of your work on a project
 - Switch between versions instantly
 - Make comparisons between versions
 - Develop multiple versions simultaneously
 - Allows for coordinated collaborations in team projects

Why you need version control

- You'll make mistakes in design and implementation.
- You'll employ incremental development
- You'll want to have a backup system
- You'll want to make experimental changes, and revert them.
- Every professional software developer uses it.